FROM COMPONENTS TO SYSTEMS

Design and Development of an Interactive Electronics Learning Game

PROJECT{“A GAME”} REPORT

Submitted By

[Dhruv Rathore]

Game Development Team

Project Type
Interactive Educational Game / Game Development Project

Project Domain
Game Development • Interactive Learning • Electronics & Device Exploration

Project Initiation

Idea Conceived: 01 August 2026
Project Preparation & Execution Started: 05 August 2026

Technologies

Unreal Engine 5 • Blueprints • C++ • Blender • UMG • Git

“Explore. Build. Connect. Understand.”

TABLE OF CONTENTS

1. Subject	3
2. Concept	3
3. Idea	5
4. Technologies	6
5. Coordination of Technology with the Project Concept	20
6. Structure	24
7. Flowchart	34
8. UI / User Interaction	39
9. Backend	40
10. Working Model	44
11. Testing	50
12. Implementation and Integration	55
13. Testing Stage 2 — Re-Testing and Correction Testing	60
14. Updation	64
15. User Interaction Test	69
16. Feedback	75
17. Conclusion	83

1. SUBJECT

Subject:
Design and Development of an Interactive Electronics Learning Game

Project Title:
From Components to Systems

Project Type:

Electronics Simulation Game

---

2. CONCEPT

The core concept of the project is to transform **electronics and electronic-device learning into an interactive, practical and progressive gameplay experience**.

The proposed game is designed to help users understand **how everyday electrical and electronic devices are built internally, how their different components are connected, and how these components work together to perform the functions of the complete device**.

Rather than presenting electronics as a purely theoretical subject, the game provides an interactive environment in which users can explore, assemble and connect the internal parts of different devices and observe how the complete system responds.

The learning experience begins with simple and familiar electrical components and gradually develops toward more complex electronic systems. The player can interact with components, identify their positions, establish connections and observe the relationship between individual parts and the overall functionality of a device.

The concept is not limited to teaching individual components in isolation. Instead, the primary focus is on understanding the **relationship between components, circuits, functional modules and complete electronic devices**.

For example, the player may initially interact with simple elements such as:

- Power sources and basic electrical connections
- Switches and control mechanisms
- Lamps, LEDs and other output components
- Motors and simple mechanical-electrical systems
- Basic circuit boards and electronic components

As the learning experience progresses, these concepts are applied to increasingly complex real-world devices. The player can explore how different sections and components are organized inside devices such as **electric lamps, fans, remote controls, televisions, refrigerators and other electronic appliances**.

The game therefore focuses on answering practical questions such as:

- What components are present inside a device?
- Where are these components located?
- How are different components connected?
- What role does each component or section perform?
- How do multiple components work together to produce the final function of the device?
- What happens when a connection or component is incorrect?
- How can a user identify and correct a problem within a system?

The ultimate direction of the concept is to gradually develop the player's understanding from **simple electrical connections and small functional systems toward complex electronic architectures**, eventually leading to computer hardware and motherboard-level systems.

The game will therefore establish a conceptual progression from:

> **Component → Connection → Circuit → Functional Module → Electronic Device → Complex System → Computer Hardware**

The fundamental philosophy of the game is:

> **Learn by building, connecting, exploring, observing and solving.**

Instead of simply explaining how an electronic device works, the game allows the player to **interact with its internal structure and discover its working through practical experimentation**.

The overall objective is to make electronics easier and more engaging to understand by converting the process of exploring and assembling electronic systems into an interactive game-based learning experience.

---

3. IDEA

The initial idea for the project was conceived on **01 August 2026**.

The idea originated from a simple observation: children and beginners often enjoy assembling small objects and experimenting with how individual parts work together.

This curiosity can be converted into an educational experience by providing a virtual environment where users can safely experiment with electronic components.

The initial gameplay will focus on a **Drag-and-Drop + Connect** interaction model.

A player may receive several components and a target objective.

For example:

Objective:
"Turn ON the bulb."

The player may be provided with:

- Battery
- Wires
- Switch
- Bulb
- Resistor

The player must determine how the components should be connected.

If the circuit is correctly constructed:

Circuit Complete → Current Flow → Bulb Turns ON → Level Completed

If the connection is incorrect:

Incorrect Connection → Circuit Does Not Operate → Feedback → Player Tries Again

The game will contain approximately **100 progressive levels**, with each level introducing a new concept, component or system.

The final levels will move beyond individual components and basic circuits toward the understanding of complex electronic assemblies, with the long-term goal of allowing the player to progressively construct and understand a **computer motherboard/system architecture**.

The project preparation and execution phase officially started on **05 August 2026**.

4. TECHNOLOGIES

The proposed game requires a technology stack capable of supporting interactive 3D environments, component manipulation, drag-and-drop interaction, device assembly, visual feedback, progressive gameplay and future expansion toward complex electronic-system simulations.

The technology stack has been selected with three major objectives:

1. **Ease of learning and development**
2. **Minimum initial development cost**
3. **Ability to expand the project in future**

For the initial development stage, the project can be developed using primarily free tools. The first playable prototype, including the initial set of levels, does not require a paid backend, paid database or paid development software.

4.1 Technology Status Symbols

Symbol	Meaning
🟢	Free / Open Source
🟡	Free plan or free under specific conditions
🔴	Paid / Not required initially
⭐	Essential for the current project
🧠	Relatively easy to learn for this project

4.2 Core Technology Stack

Technology	Status	Difficulty	What It Is	What It Will Create in the Game
⭐ Unreal Engine 5	🟡 Free under current game-development terms	🧠🧠	Main Game Engine	Complete game, levels, 3D world, physics, interaction and simulation
⭐ Blueprints	🟢 Included with Unreal Engine	🧠	Visual programming system	Drag-and-drop, connections, level logic, device behavior and gameplay
⭐ C++	🟢 Free language/toolchain	🧠🧠🧠	Programming language	Advanced game systems and complex logic
⭐ UMG	🟢 Included with Unreal Engine	🧠	Unreal's UI system	Menus, buttons, instructions, level information and feedback
⭐ Blender	🟢 Free & Open Source	🧠🧠	3D modelling software	Electronic components, device parts and 3D

Technology	Status	Difficulty	What It Is	What It Will Create in the Game
				assets
⭐ Git	🟢 Free & Open Source	🧠🧠	Version control system	Project history and safe development
⭐ GitHub	🟡 Free plan	🧠	Online code/project repository	Backup, collaboration and version management
Figma	🟡 Free Starter plan	🧠	UI/UX design tool	UI layouts, menus and interface prototypes
Audacity	🟢 Free & Open Source	🧠	Audio editing software	Sound effects and audio preparation
Data Tables / Data Assets	🟢 Included with Unreal Engine	🧠🧠	Structured game data system	Component information, device information and level configuration
SaveGame	🟢 Included with Unreal Engine	🧠🧠	Local save system	Player progress and unlocked levels
Visual Studio Community	🟢 Free for individual development	🧠🧠	C++ development environment	Writing, compiling and debugging C++ code

4.3 Unreal Engine 5

Status: 🟡 Free under applicable game-development terms

Importance: ⭐ Essential

Learning Difficulty: 🧠🧠

Unreal Engine 5 will serve as the **primary development environment** for the entire project.

It will be responsible for bringing all other technologies together and converting the project concept into an actual playable game.

The engine will be used for:

- Creating game levels
- Creating the interactive environment
- Importing 3D models
- Handling player interaction
- Managing physics
- Creating component interaction
- Managing game states
- Creating visual effects
- Implementing audio

- Creating menus and interfaces
- Testing gameplay
- Packaging the final game

In simple terms:

> **Unreal Engine = The main workshop where the complete game will be built.**

For example:

Blender creates a virtual bulb → Unreal imports the bulb → Blueprint gives the bulb behavior → UMG displays instructions → Unreal runs the complete gameplay.

Epic currently provides Unreal Engine for game developers without an upfront engine fee under its standard game-development terms; royalties apply after the first $1 million USD in lifetime gross product revenue for a royalty-bearing game.

4.4 Blueprints

Status: 🟢 Included with Unreal Engine

Importance: ⭐ Essential

Learning Difficulty: 🧠 Easy for initial development

Blueprints are Unreal Engine's **visual scripting system**.

Instead of writing code for every small gameplay feature, logic can be created by connecting visual nodes.

Epic describes Blueprints as a node-based system for creating gameplay functionality inside Unreal Engine.

What we will create with Blueprints

Blueprints will initially handle most of the game's interactive behavior.

For example:

```
Player selects component
        ↓
Component follows mouse
        ↓
Player places component
        ↓
Player connects component
        ↓
Game checks connection
        ↓
Correct?
   ↙          ↘
 YES           NO
```

```
 ↓                ↓
Activate        Feedback
device          / Retry
```

Blueprints can therefore be used for:

- Drag-and-drop
- Component selection
- Component placement
- Connection detection
- Correct/incorrect connection
- Device activation
- Level completion
- Level progression
- Tutorial logic
- Feedback
- Interaction effects

Why Blueprints are important for this project

Because the first version of the game is highly interaction-based, Blueprints allow the core gameplay to be developed without writing large amounts of C++.

Therefore:

> **Blueprints will be the primary gameplay-development technology in the initial phase.**

4.5 C++

Status: 🟢 Free

Importance: ⭐ Important, but not required for every feature

Learning Difficulty: 🧠🧠🧠

C++ will be used for systems where structured programming, reusable architecture or more advanced logic is beneficial.

C++ will not be used unnecessarily for simple gameplay features.

Potential C++ responsibilities include:

- Core component system

- Advanced connection validation
- Complex device logic
- Reusable gameplay systems
- Advanced simulation logic
- Performance-sensitive operations
- Communication between major game systems

Recommended development approach

The project will follow:

> **Blueprint First → C++ Where Required**

This approach will make development easier while still allowing the project to use C++ where it provides real benefits.

4.6 Unreal Motion Graphics (UMG)

Status: 🟢 Included with Unreal Engine

Importance: ⭐ Essential

Learning Difficulty: 🧠 Easy

UMG, or Unreal Motion Graphics, will be used to create the game's user interface.

Epic's documentation identifies UMG as the system for creating interface elements such as HUDs, menus, buttons and other in-game UI components.

UMG will create:

- Main Menu
- Start button
- Level selection
- Objective panel
- Component information
- Instructions
- Progress indicator
- Success message
- Error message
- Reset button

- Pause menu
- Completion screen

For example:

```
LEVEL 01

Objective: Make the lamp work

          [ GAME WORKSPACE ]


Components:
[Battery] [Switch] [Wire] [Lamp]

              [ TEST ]
```

4.7 Blender

Status: 🟢 Free & Open Source

Importance: ⭐ Essential

Learning Difficulty: 🧠🧠

Blender will be used to create and modify the 3D assets required for the game.

Blender is officially distributed as free and open-source software under the GNU GPL.

Blender will create:

- Bulbs
- Switches
- Wires
- Fans
- Remote-control parts
- Circuit boards
- Electronic components
- Device housings
- TV components
- Refrigerator components
- Connectors
- Motherboard components

- Environment objects

Workflow

```
Blender
   ↓
Create / Modify 3D Component
   ↓
Export
   ↓
Unreal Engine
   ↓
Add Interaction
   ↓
Playable Component
```

For the first prototype, we should **not spend excessive time making photorealistic models**.

Simple, clean and recognizable models are sufficient.

4.8 Figma

Status: 🟡 Free Starter Plan

Importance: Optional but useful

Learning Difficulty: 🧠 Easy

Figma will be used primarily for planning the game's interface before implementing it inside Unreal Engine.

The current Figma Starter plan is free, although it has limitations compared with paid plans.

Figma will help create:

- Main menu design
- Level-selection design
- Buttons
- Icons
- Instruction panels
- Component panels
- Feedback screens
- UI prototypes

Workflow

```
Idea
 ↓
Figma UI Design
 ↓
UI Prototype
 ↓
Unreal UMG
 ↓
Actual Game Interface
```

Figma is **not essential for making the game run**. If required, UI can also be designed directly inside Unreal.

4.9 Git

Status: 🟢 Free & Open Source

Importance: ⭐ Recommended

Learning Difficulty: 🧠🧠

Git will be used for **version control**.

Game development involves frequent changes. A feature may work today and break tomorrow. Git allows earlier versions of the project to be tracked and restored.

Example

```
Version 1
   ↓
Basic Drag & Drop
   ↓
Version 2
   ↓
Connection System
   ↓
Version 3
   ↓
Level System
   ↓
Version 4
   ↓
UI + Save System
```

If a major change causes problems, previous versions can be recovered.

4.10 GitHub

Status: 🟡 Free Plan

Importance: ⭐ Recommended

Learning Difficulty: 🧠🧠

GitHub will be used as an online repository for source code and project version management.

GitHub currently provides a free personal plan with unlimited public and private repositories, subject to plan limits.

Important limitation for Unreal projects

Unreal projects can contain large binary assets. Therefore, GitHub should **not be treated as unlimited cloud storage for the entire game**.

GitHub's current included Git LFS allowance is limited, so large Unreal assets should be managed carefully.

For the initial development phase, GitHub can primarily be used for:

- C++ source code
- Blueprint-related project files where appropriate
- Configuration
- Documentation
- Version tracking

Large generated folders should be excluded using `.gitignore`.

4.11 Audacity

Status: 🟢 Free & Open Source

Importance: Optional

Learning Difficulty: 🧠 Easy

Audacity will be used to create and edit sound effects.

Audacity is officially available as free, open-source software.

Possible uses:

- Button click sounds
- Connection sound

- Success sound
- Error sound
- Device activation sound
- Electrical hum
- Motor/fan sound
- Notification sounds

The final audio playback will be handled inside Unreal Engine.

4.12 Data Tables / Data Assets

Status: 🟢 Included with Unreal Engine

Importance: ⭐ Essential for scalable development

Learning Difficulty: 🧠🧠

These systems will be used to store structured information about components, devices and gameplay content.

Instead of hardcoding every object individually, the game can store information such as:

```
Component ID
Component Name
Component Type
Model
Description
Connection Points
Required Connections
Function
Visual State
Sound
```

For example:

```
ID: LAMP_001
Name: Electric Lamp
Type: Output Device
Input: Power
State: OFF / ON
```

This architecture will make it easier to add new devices and gameplay content later.

4.13 SaveGame

Status: 🟢 Included with Unreal Engine

Importance: ⭐ Recommended

Learning Difficulty: 🧠🧠

The Unreal Engine SaveGame system will initially be used for local player progress.

It can store:

- Completed levels
- Unlocked levels
- Best score
- Completed tutorials
- Settings
- Game progress

Initial architecture

```
Player
 ↓
Complete Level
 ↓
Save Progress
 ↓
SaveGame
 ↓
Local Computer
```

A cloud database is not required for this functionality.

4.14 Visual Studio Community

Status: 🟢 Free for individual development

Importance: ⭐ Recommended for C++

Learning Difficulty: 🧠🧠

Visual Studio Community will be used as the C++ development environment when C++ is required.

Microsoft currently lists Visual Studio Community as a free development product and specifically describes it as an IDE for C++ developers on Windows.

It will be used for:

- Writing C++

- Compiling C++
- Debugging
- Managing Unreal C++ classes
- Reading compiler errors
- Building advanced game systems

4.15 Backend

Initial Status: 🟢 Not Required

A separate backend will **not be implemented in the first version**.

The initial game can operate as an offline application.

The required systems can be handled locally:

```
Unreal Engine
       │
       ├── Gameplay
       ├── Components
       ├── Device Logic
       ├── Levels
       ├── UI
       ├── Validation
       └── SaveGame
```

A backend may be introduced in a future version if the project requires:

- User accounts
- Cloud saves
- Online leaderboards
- Teacher/student dashboards
- Online progress tracking
- Analytics
- Multiplayer features

Possible future technology:

Supabase / Firebase / Custom Backend

These technologies are therefore **not part of the initial required stack**.

4.16 What Technology Creates What?

The relationship between the technologies can be summarized as follows:

Requirement	Technology
Complete game	⭐ Unreal Engine
Gameplay logic	⭐ Blueprints
Advanced programming	C++
Game interface	UMG
3D components	Blender
UI planning	Figma
Sound editing	Audacity
Component/level data	Data Tables / Data Assets
Player progress	SaveGame
C++ development	Visual Studio Community
Version control	Git
Online repository	GitHub
Online backend	Not required initially

4.17 Recommended Initial Stack

For the **first playable version**, the project should be kept as simple as possible.

Required

⭐ **Unreal Engine 5**
⭐ **Blueprints**
⭐ **UMG**
⭐ **Blender**
⭐ **Visual Studio Community**
⭐ **C++**
⭐ **Git**

Helpful but not mandatory

Figma
GitHub
Audacity

Not required initially

❌ Node.js
❌ Express.js
❌ MongoDB

- ❌ PostgreSQL
- ❌ Supabase
- ❌ Firebase
- ❌ AWS
- ❌ Paid APIs
- ❌ Paid development software

4.18 Initial Cost Strategy

The initial prototype will be developed with a **zero-software-cost strategy**.

For the first development milestone, the project can use:

Unreal Engine + Blueprints + C++ + UMG + Blender + Visual Studio Community + Git

without purchasing paid development software.

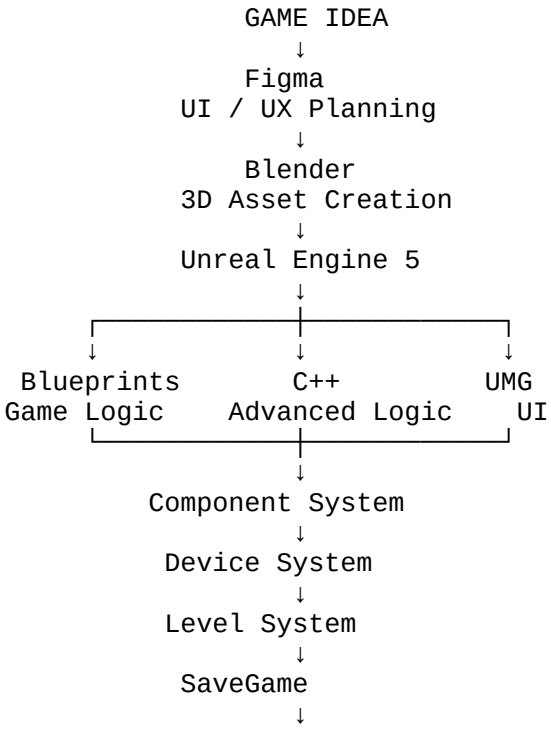
Figma and GitHub can be used through their free plans, while Unreal Engine's game-development licensing remains free under the applicable revenue threshold.

Therefore, the initial objective is:

> **Build the first playable version first; introduce paid services only when the project genuinely requires them.**

4.19 Technology Development Pipeline

The complete technology pipeline will be:



```
Testing
   ↓
Final Game
```

Final Technology Principle

The project will follow a **Blueprint-first and modular development approach**.

Simple gameplay functionality will be implemented through Blueprints, while C++ will be introduced only where advanced or reusable systems require it. 3D assets will be created or prepared using Blender, UI will be implemented using UMG, and structured game information will be maintained through Unreal's data systems.

This approach minimizes development complexity while keeping the project scalable for future expansion.

---

5. COORDINATION OF TECHNOLOGY WITH THE PROJECT CONCEPT

The selected technologies work together to convert the project's concept of **interactive electronic-device learning** into a playable game experience.

Each technology has a specific responsibility within the development process.

Unreal Engine 5 — Game Environment

Unreal Engine 5 will act as the central development platform where all game systems will be integrated.

It will manage:

- Game environment
- Interactive objects
- Device components
- Player interaction
- Physics and movement
- Levels
- Visual effects
- Audio
- Game states

In simple terms:

Unreal Engine is the main environment in which the complete game will run.

Blueprints — Gameplay and Interaction

Blueprints will be primarily responsible for the interactive behavior of the game.

They will be used for:

- Selecting components
- Drag-and-drop interaction
- Placing components
- Connecting components
- Detecting correct and incorrect connections
- Activating device functions
- Showing success or failure feedback
- Level progression
- Tutorial and interaction logic

For example:

Select Component → Drag → Place → Connect → Validate → Device Responds

This makes Blueprints particularly suitable for the initial prototype.

C++ — Advanced Game Systems

C++ will be introduced where more structured or reusable programming is required.

It may be used for:

- Core game systems
- Advanced interaction logic
- Complex validation
- Reusable component architecture
- Advanced simulation systems
- Performance optimization

The initial development will follow a **Blueprint-first approach**, with C++ being introduced where it provides a practical advantage.

Blender — Electronic Device Assets

Blender will be used to create and prepare the 3D models required by the game.

These may include:

- Electronic components

- Device parts
- Circuit boards
- Switches
- Motors
- Fans
- Remote-control components
- Television components
- Refrigerator components
- Computer and motherboard components

The models created in Blender will be imported into Unreal Engine and converted into interactive game objects.

UMG — User Interface

Unreal Motion Graphics (UMG) will be used to create the interface through which the player interacts with the game.

It will provide:

- Instructions
- Objectives
- Component panels
- Buttons
- Level information
- Progress indicators
- Feedback messages
- Completion screens

Data Tables / Data Assets — Game Content Management

Unreal's data systems will be used to organize information about components, devices and levels.

For example, a device can contain information such as:

Device Name → Required Components → Connection Rules → Expected Output → Completion Condition

This approach allows new components and devices to be added without redesigning the entire game system.

SaveGame — Player Progress

The Unreal Engine SaveGame system will be used to store local player progress.

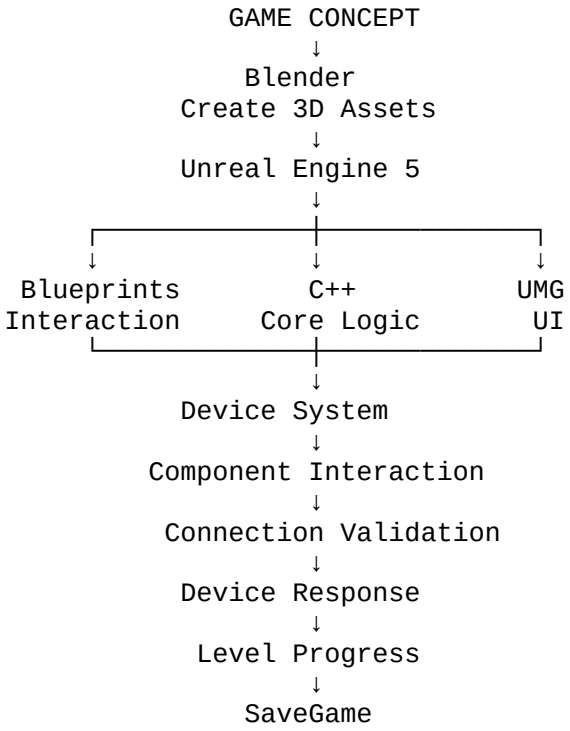
It can maintain information such as:

- Completed levels
- Unlocked content
- Player progress
- Settings
- Best results

A separate online backend will not be required for the initial version.

Overall Technology Coordination

The technologies will work together through the following development pipeline:



The overall development philosophy is to keep the initial implementation **simple, modular and easy to expand**.

The technology stack therefore supports the project's central learning model:

Explore → Assemble → Connect → Observe → Understand → Solve

This coordination allows the project to evolve from a simple interactive prototype into a larger educational game covering increasingly complex electronic devices and systems.

---

6. STRUCTURE

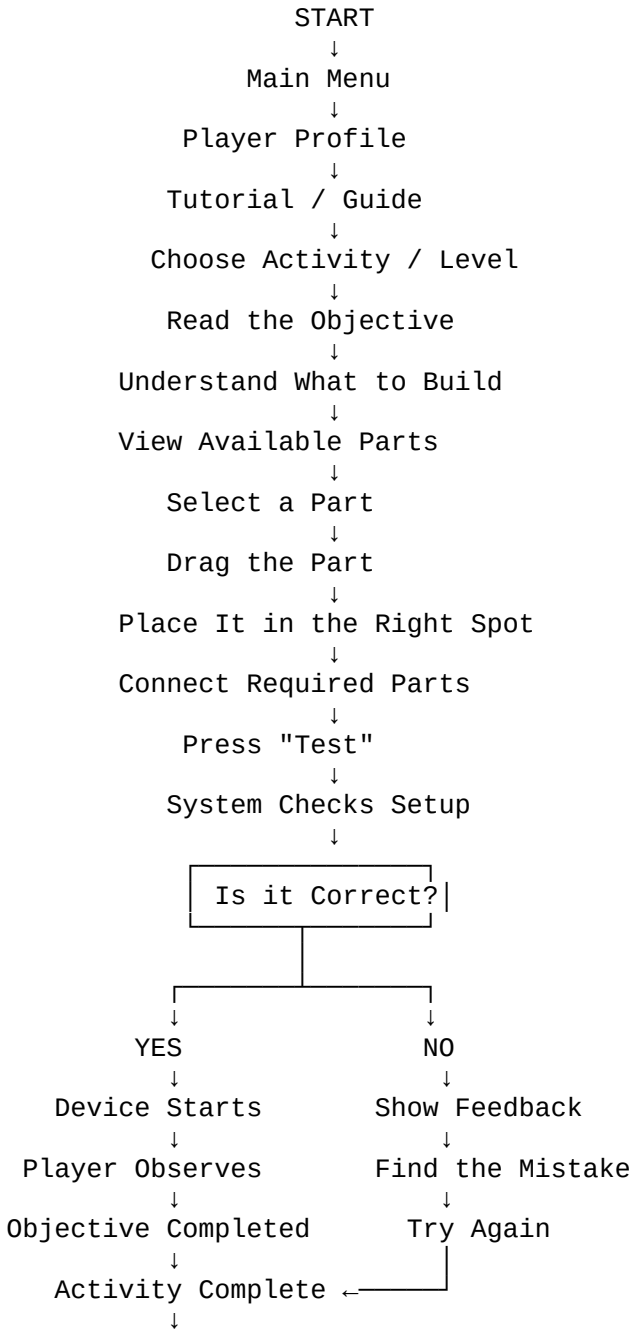
The game will follow a **progressive, modular and interactive learning structure**. The player will gradually move from simple electrical connections to understanding how different parts work together inside increasingly complex devices and systems.

The structure is designed around the following learning progression:

> **Simple Parts → Simple Connections → Working Functions → Small Devices → Everyday Devices → Complex Devices → Larger Systems**

6.1 Overall Game Structure

The complete game flow will be organized as follows:



```
Save Progress
      ↓
Unlock Next Activity
```

Simple Gameplay Process

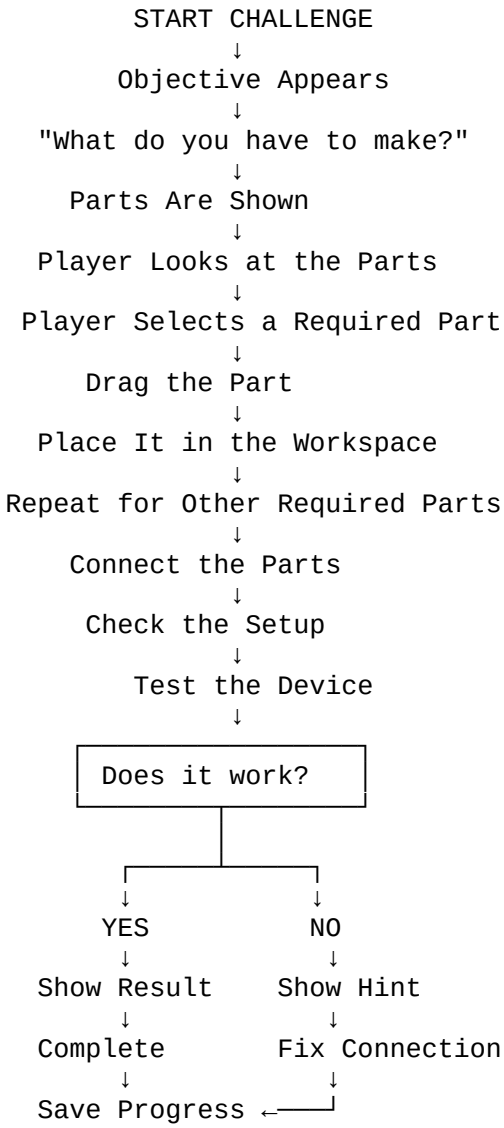
The player will basically follow:

See → Understand → Pick → Place → Connect → Test → Observe → Correct → Complete

This simple interaction cycle will remain consistent throughout the game, while the complexity of the tasks will increase progressively.

6.2 Player Interaction Structure

The player's interaction with a challenge will be divided into small and understandable steps.



Example

A very simple challenge could work like this:

```
Objective:
```

```
"Make the light turn ON."
        ↓
Parts shown:
Battery + Wire + Switch + Bulb
        ↓
Player picks the parts
        ↓
Places them
        ↓
Connects them
        ↓
Presses TEST
        ↓
Light turns ON
        ↓
Challenge Complete
```

The same basic interaction can later be applied to much more complex devices.

6.3 Progressive Learning Structure

The game will not introduce complex devices immediately. The learning experience will gradually increase in complexity.

Stage 1 — Understanding Basic Parts

The player is introduced to simple and easily recognizable parts.

Examples:

- Battery
- Wire
- Bulb
- Switch
- Small light

The main purpose is to teach the player:

- What a part looks like
- Where it can be placed
- How it can be connected
- What happens when it is connected correctly

The focus is on **learning the basic interaction system**.

Stage 2 — Making Simple Things Work

The player starts combining different parts to create a working function.

Examples:

- Turning a light ON
- Starting a small motor

- Making a fan rotate
- Using a switch to control something

The focus changes from:

> **“What is this part?”**

to:

> **“How do these parts work together?”**

Stage 3 — Building Simple Devices

The player begins working with small devices that contain multiple parts.

The player may have to:

- Find the required parts
- Place them correctly
- Connect them
- Test the setup
- Identify mistakes
- Correct the setup

The goal is to help the player understand that **a working device is created by combining multiple parts in the correct way**.

Stage 4 — Understanding Everyday Devices

The game then moves toward devices that players commonly see and use in daily life.

Examples can include:

- Electric lamp
- Fan
- Remote control
- Television
- Refrigerator
- Other household electronic devices

Instead of simply showing the finished device, the player can explore its internal arrangement and understand how different sections contribute to its operation.

The focus becomes:

> **“What is inside this device and how do its different parts work together?”**

Stage 5 — Working With Larger Systems

As the player progresses, individual devices become more complex.

A single device may contain several different sections, and each section may have its own function.

The player may need to understand:

- Which part performs which job
- Which parts need to work together
- How one section affects another
- What happens when something is connected incorrectly
- How to find and fix a problem

The learning process therefore moves from:

```
One Part
   ↓
Several Parts
   ↓
One Function
   ↓
Several Functions
   ↓
One Complete Device
   ↓
Multiple Connected Sections
```

Stage 6 — Advanced System Understanding

In the later stages, the player works with larger and more complicated systems.

Instead of solving only one simple connection, the player may need to understand the relationship between multiple sections and their functions.

The player gradually develops the ability to think about:

> **Input → Internal Process → Output**

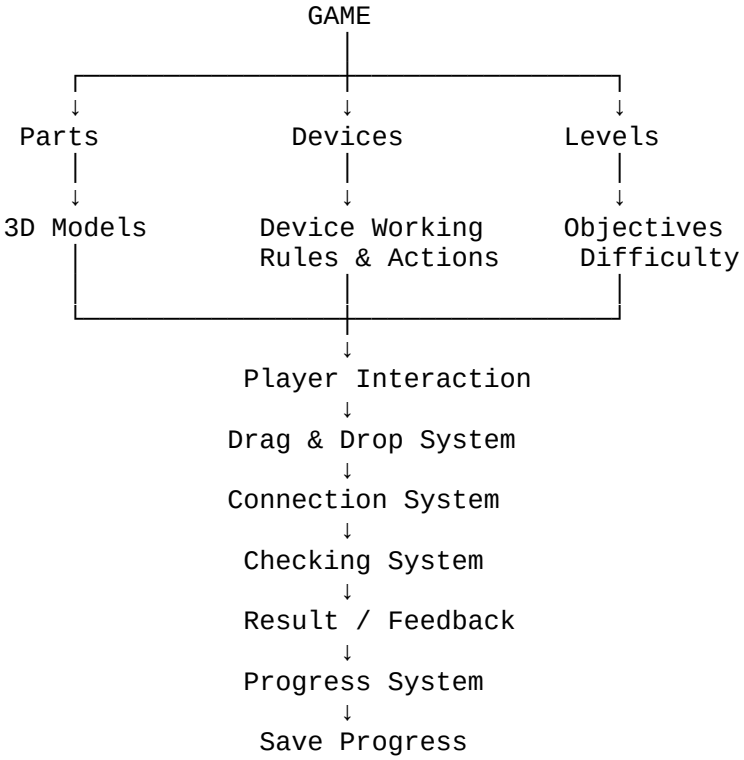
and

> **Part → Section → System**

The final stages can therefore represent highly complex electronic systems without requiring the Structure section to specify a particular final device.

6.4 Modular Game Structure

The game will be developed in separate but connected systems so that new content can be added easily.



Simple Explanation

Each system will have a clear responsibility:

System	Simple Purpose
Parts	Stores the objects the player can use
Devices	Defines how a device should work
Levels	Defines what the player has to do
Player Interaction	Allows the player to pick and move objects
Drag & Drop	Allows objects to be placed
Connection System	Checks how objects are connected
Checking System	Determines whether the setup is correct
Feedback System	Shows what went right or wrong
Progress System	Tracks completed activities
Save System	Saves player progress

This modular approach will make it possible to add new devices and challenges without rebuilding the complete game.

6.5 Device Interaction Structure

Each device will have its own internal structure and working rules.

The basic structure will be:

```
            DEVICE
              ↓
     Different Parts
              ↓
     Parts Have Roles
              ↓
Parts Are Connected
              ↓
Different Sections Work
              ↓
 Device Receives Input
              ↓
 Device Processes Input
              ↓
   Device Gives Output
```

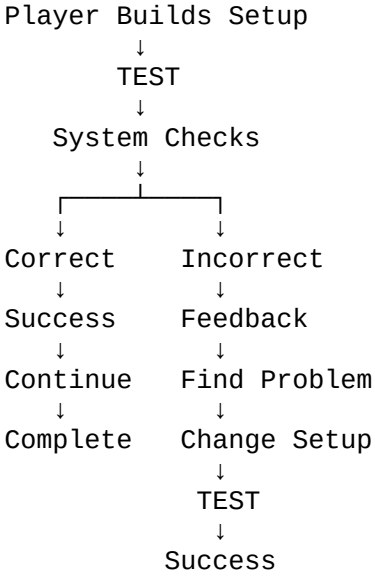
For example:

```
 Press a Button
       ↓
Internal Action
       ↓
Signal Travels
       ↓
Required Section Works
       ↓
 Desired Result
```

This allows the player to understand the **working relationship between different parts** instead of simply memorizing component names.

6.6 Error and Feedback Structure

An important part of the game will be allowing players to make mistakes and learn from them.



Feedback should be simple and understandable.

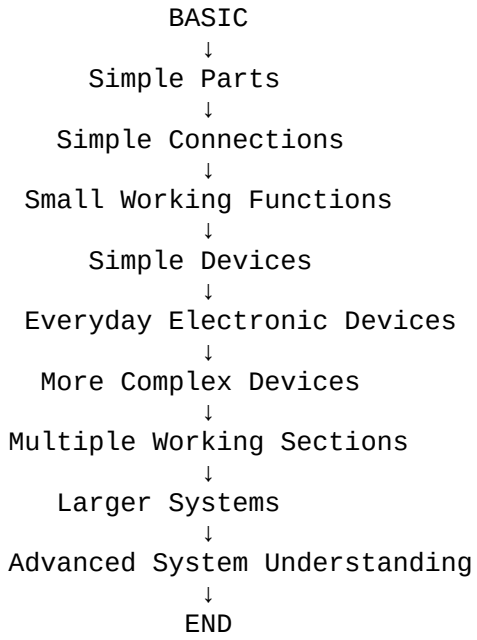
For example:

> **“The connection is incomplete.”**
>
> **“This part is connected to the wrong point.”**
>
> **“Check the power connection.”**

This makes the game more educational than simply displaying **“Wrong Answer.”**

6.7 Content Progression

The complete content progression will follow a gradual increase in complexity:



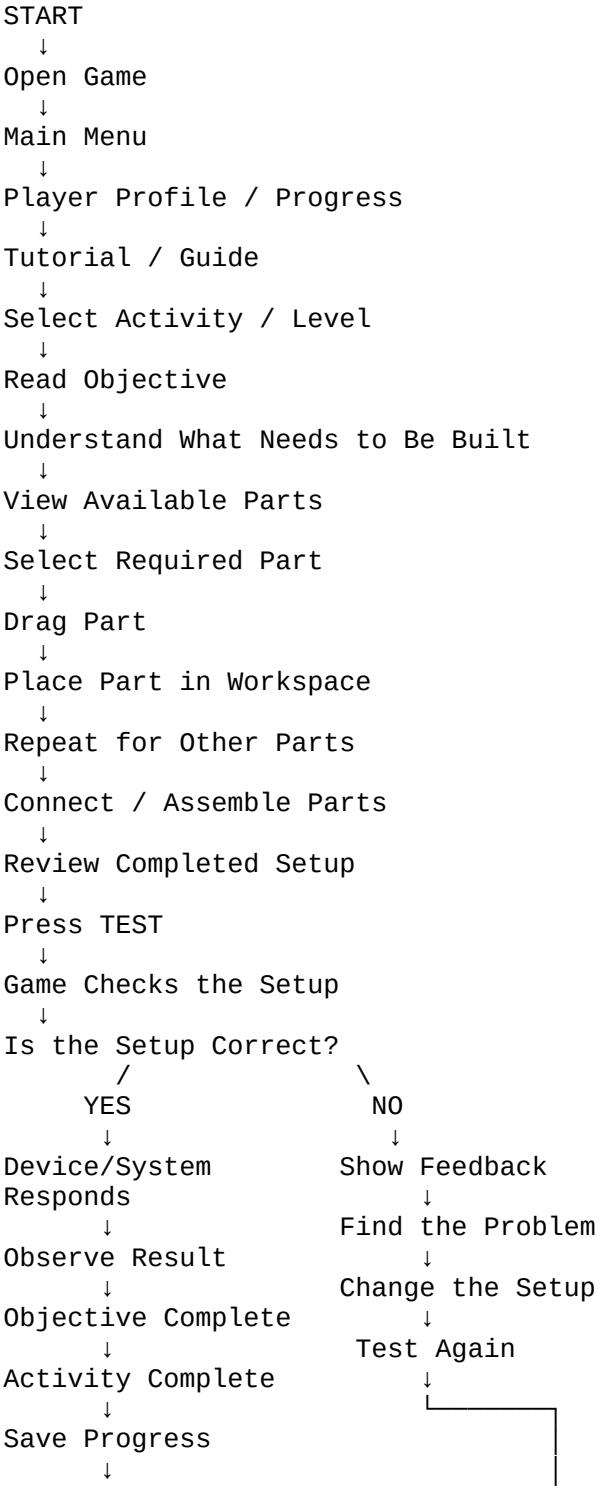
---

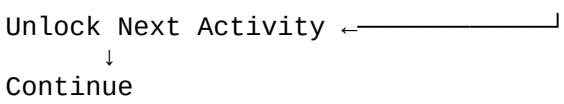
7. FLOWCHART

The game flowchart represents the complete sequence of interaction between the player and the game system.

The flow is designed to remain simple at the beginning and become more detailed internally as the player works with increasingly complex devices and systems.

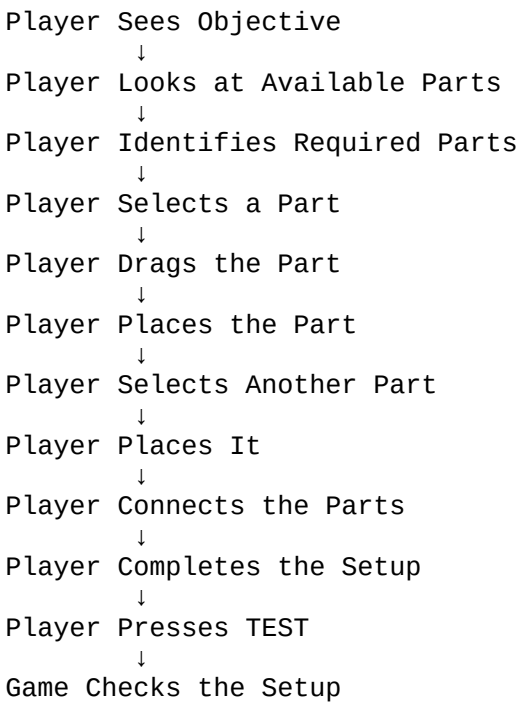
7.1 Main Gameplay Flow





7.2 Player Interaction Flow

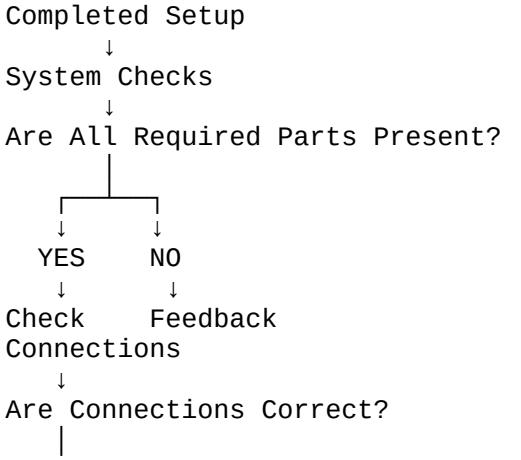
The interaction system will follow a simple sequence so that beginners can understand the gameplay easily.

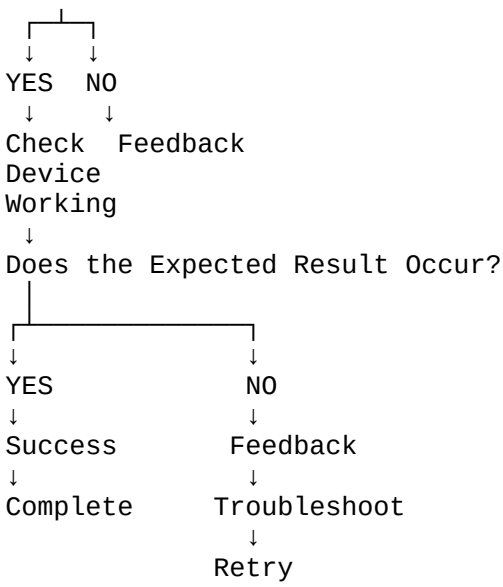


This process allows the player to learn through direct interaction rather than through lengthy theoretical instructions.

7.3 System Decision Flow

After the player completes an arrangement, the system checks whether the required conditions have been satisfied.





This allows the system to evaluate the challenge step by step rather than treating the entire setup as simply **right or wrong**.

7.4 Success Flow

When the player's setup is correct, the corresponding object, device or system will respond according to the challenge.

```
Correct Setup
      ↓
System Validates
      ↓
Required Function Activates
      ↓
Player Observes Result
      ↓
Success Message
      ↓
Objective Completed
      ↓
Activity Completed
      ↓
Progress Saved
      ↓
Next Activity Unlocked
```

Examples of visible responses may include:

```
Correct Connection
      ↓
Lamp Turns ON
```

```
Correct Setup
      ↓
Fan Starts Rotating
```

```
Correct Assembly
      ↓
Remote Performs the Expected Action
```

```
Correct Device Setup
      ↓
Device Function Becomes Active
```

The visual response helps the player understand the relationship between the setup and the resulting function.

7.5 Error and Retry Flow

The game will treat mistakes as part of the learning process.

```
Player Tests Setup
      ↓
Setup Incorrect
      ↓
Identify Problem
      ↓
Show Simple Feedback
      ↓
Highlight / Explain Problem
      ↓
Player Changes Part
      ↓
Player Changes Connection
      ↓
Test Again
      ↓
System Checks Again
      ↓
Correct?
   /       \
 YES        NO
  ↓          ↓
Success    More Feedback
```

The purpose of this system is to encourage **problem-solving and experimentation** instead of penalizing the player for every incorrect attempt.

7.6 Complete Learning Flow

The complete educational gameplay cycle can be summarized as:

```
Explore
   ↓
Understand
   ↓
Select
   ↓
Place
   ↓
Connect
   ↓
Assemble
   ↓
Test
```

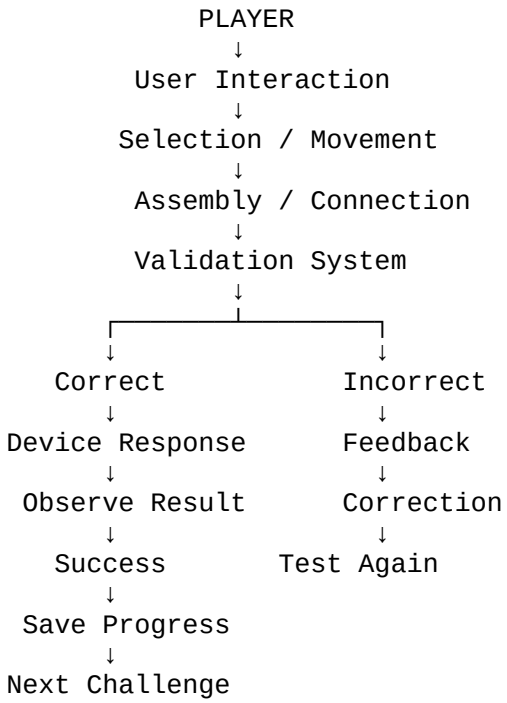
```
    ↓
Observe
    ↓
Find Mistakes
    ↓
Correct
    ↓
Complete
    ↓
Save Progress
    ↓
Move to a More Complex Challenge
```

This cycle will remain the foundation of the game throughout its development.

The difference between early and advanced challenges will be the **complexity of the parts, number of connections, number of functional sections and relationships between those sections**, rather than a change in the basic gameplay interaction.

7.7 High-Level System Flow

The complete relationship between the player, game system and device response can be represented as:



The flowchart therefore represents the central operational principle of the project:

Interact → Assemble → Test → Observe → Understand → Correct → Progress

This process will allow the game to maintain a consistent and easy-to-understand interaction model while supporting increasingly complex electronic devices and systems.

---

8. UI / USER INTERACTION

The user interface will be designed to provide a **simple, visual and beginner-friendly interaction experience**, particularly for users who may have little or no prior knowledge of electronics.

Since the primary purpose of the game is learning through interaction, the interface will avoid unnecessary technical information and instead guide the player through visual instructions, recognizable objects and clear feedback.

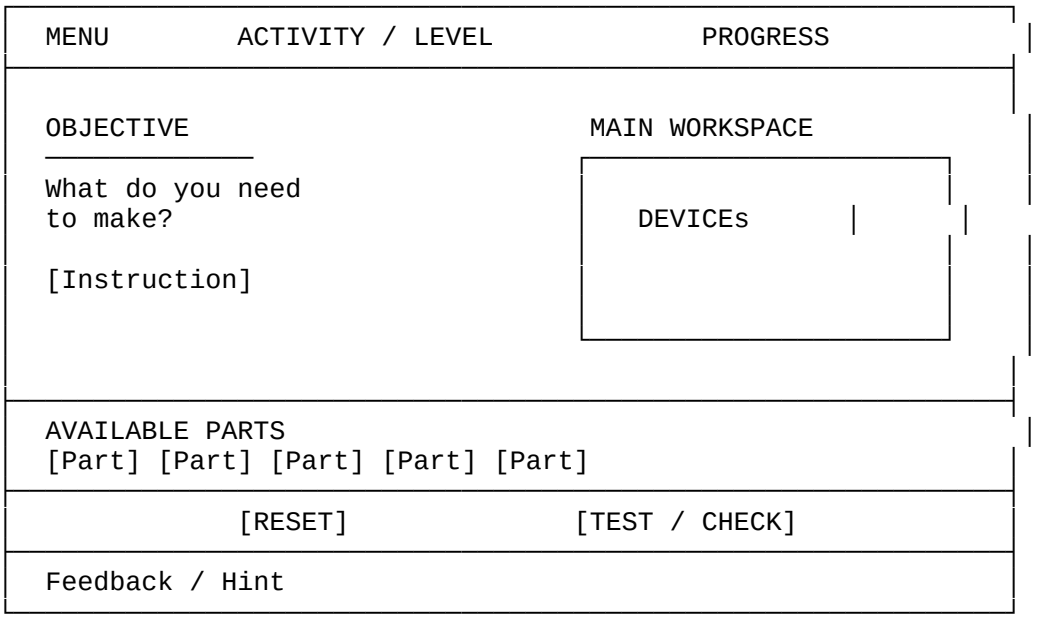
The main interaction philosophy will be:

> **See → Select → Place → Connect → Test → Observe**

8.1 Initial Interface

The initial gameplay screen will contain the following major sections:

USE HAVE TO WORK ON THIS.. Make it



The interface may contain:

- Main workspace
- Objective/instruction panel
- Available parts panel
- Device or assembly area
- Test/Check button
- Reset button

- Hint/Feedback section
- Progress indicator
- Activity or level information
- Pause/Menu controls

The exact layout may evolve during development based on usability testing.

---

9. BACKEND

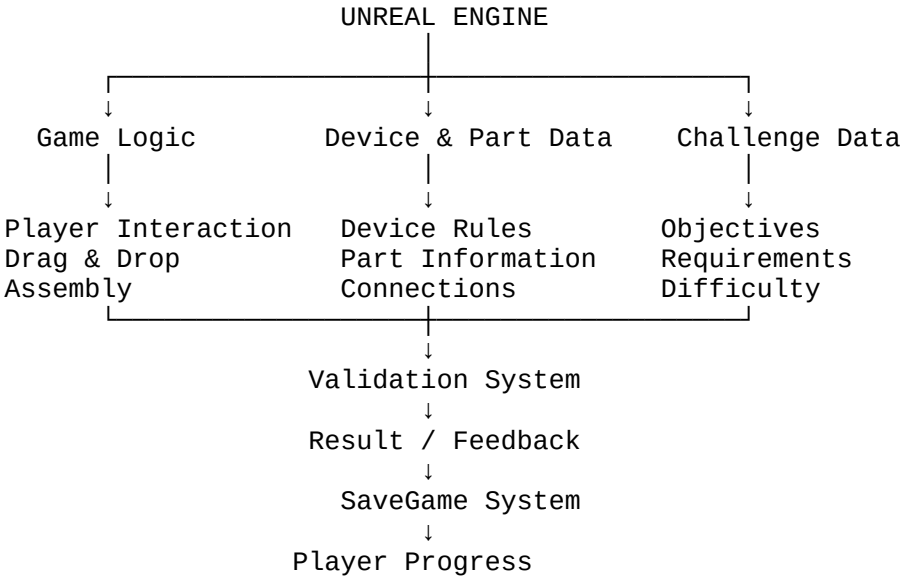
For the initial version of the project, a traditional web-based backend will **not be required**. The game can operate as a standalone application, with its primary gameplay logic, device information, challenge data and player progress managed locally through Unreal Engine.

This approach will keep the initial development **simple, lightweight and cost-effective**, while allowing a backend to be introduced later if online features become necessary.

9.1 Initial Backend Approach

The initial version will use Unreal Engine's internal systems instead of an external server.

The basic architecture will be:



This architecture is sufficient for the first playable version.

9.2 What Data Will Be Stored?

Player Data

The game can store:

- Player name/profile information
- Completed activities
- Unlocked activities
- Current progress
- Best results
- Tutorial completion
- Game settings

Part Data

Information about each available part can include:

- Part name
- Part type
- 3D/2D model
- Position
- Connection points
- Interaction rules
- Visual state
- Associated sound
- Function

Device Data

Each device can have information such as:

- Device name
- Required parts
- Part positions
- Connection rules
- Functional sections
- Expected behavior
- Success conditions
- Available hints

Challenge Data / Level

Each challenge can store:

- Objective
- Required parts
- Required arrangement
- Required connections
- Expected result
- Difficulty
- Feedback messages
- Completion condition

This information can be managed using Unreal Engine's **Data Tables / Data Assets**.

9.3 Local Save System

The player's progress will initially be stored using Unreal Engine's **SaveGame** system.

A simplified structure can be:

```
Player
   ↓
Completes Activity
   ↓
Save Progress
   ↓
SaveGame
   ↓
Local Game Storage
```

The system can remember:

```
Player Progress
├── Completed Activities
├── Unlocked Activities
├── Best Results
├── Tutorial Status
└── Settings
```

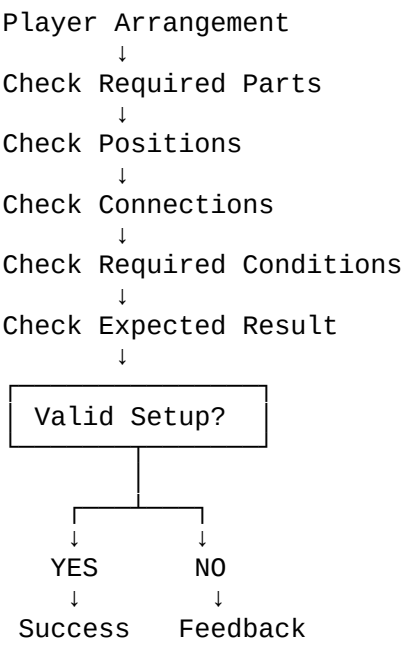
This allows the player to close the game and continue from the previous progress when the game is opened again.

9.4 Validation System

Although there is no external backend in the first version, the game will still require an internal **validation system**.

This system will determine whether the player's arrangement or assembly satisfies the requirements of the current challenge.

For example:



This validation will initially be implemented locally through **Blueprints and, where necessary, C++**.

9.5 Why an External Backend Is Not Required Initially

The first version does not require:

- User authentication
- Online accounts
- Cloud storage
- Multiplayer
- Online leaderboards
- Online matchmaking
- Teacher dashboards
- Online analytics

The recommended initial architecture is:

Unreal Engine + Local Data + SaveGame

---

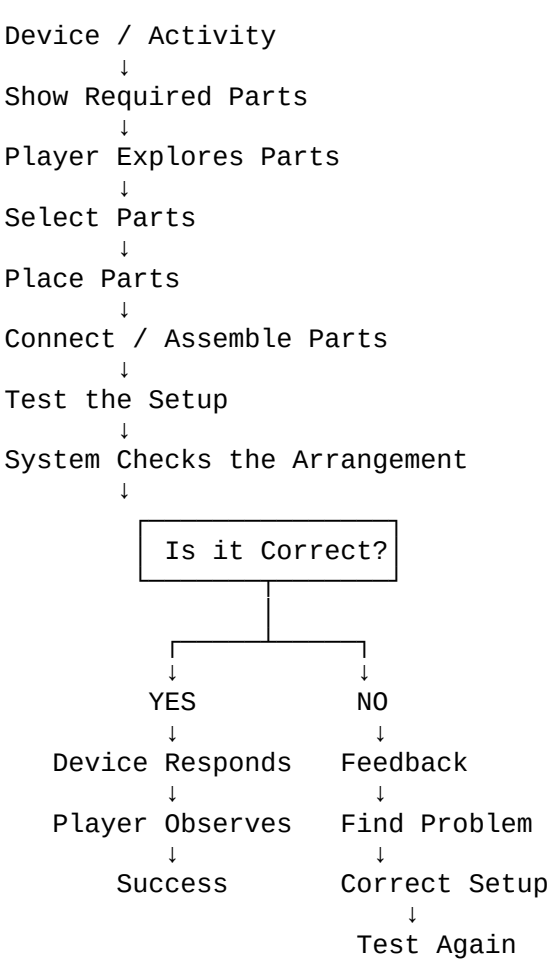
10. WORKING MODEL

The game will operate through an **interactive device assembly and simulation model** in which the player learns by exploring, placing, connecting and testing the internal parts of electrical and electronic devices.

The primary working principle is not to reproduce every real-world electrical phenomenon with complete engineering-level accuracy. Instead, the initial system will provide a **simplified and visually understandable representation of how different parts work together to perform a particular function**.

10.1 Basic Working Principle

The basic working model of the game can be represented as:



The player therefore learns through a continuous cycle of:

Explore → Assemble → Connect → Test → Observe → Correct → Understand

---

10.2 Representation of Parts

Every part used in the game will be represented as an interactive game object.

A part can have properties such as:

```
Part
├── Name
├── Type
├── Visual Model
├── Position
├── Connection Points
├── Visual State
```

For example, a simple switch may be represented as:

```
Switch
├── Name
├── ON / OFF State
├── Connection Points
├── Position
├── Visual Model
```

10.3 Representation of a Device

A complete device will be represented as a collection of connected parts and functional sections.

```
Device
├── Parts
├── Functional Sections
├── Connections
├──
├── Expected Behaviour
└── Success Condition
```

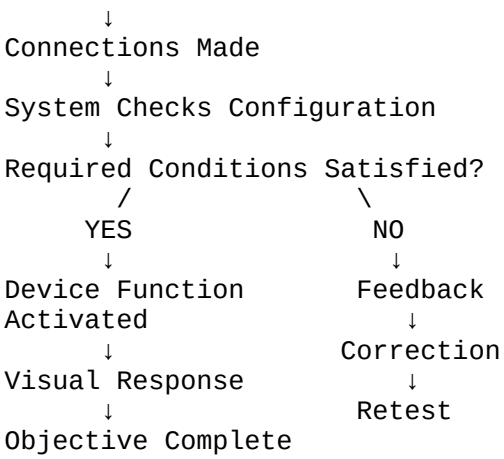
The objective is not simply to identify these objects but to understand **how their relationship produces the final function**.

10.4 Device Working Logic

Each device will have predefined rules that describe how its parts should interact.

The general logic will be:

```
Parts Available
      ↓
Required Parts Selected
      ↓
Parts Placed
```



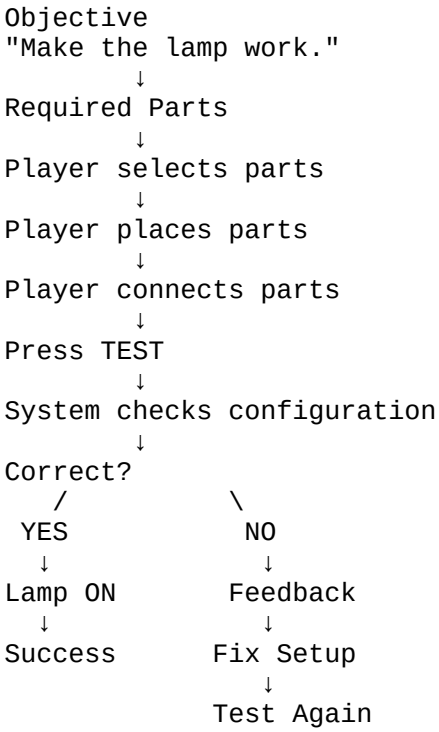
The game therefore does not need to simulate every physical process in full detail during the initial development stage.

It only needs to determine:

> **“Has the player created the correct configuration required to make this device perform its intended function?”**

10.5 Example of Simple Device Working

A basic activity may have the following structure:



The player can therefore immediately see the relationship between the assembly and the resulting output.

10.6 Working Model for More Complex Devices

As the complexity of the device increases, the same basic system can operate with multiple sections.

```
Input
  ↓
Section 1
  ↓
Section 2
  ↓
Section 3
  ↓
Output
```

Each section may contain multiple parts.

For example:

```
Device
  │
  ├── Input Section
  │
  ├── Control Section
  │
  ├── Processing / Working Section
  │
  └── Output Section
```

The player may be required to understand how these sections are related before the complete device can operate.

This creates a natural progression from:

Individual Part → Group of Parts → Functional Section → Complete Device

---

10.7 Error Handling and Troubleshooting

The working model will also support incorrect configurations.

When the player creates an incorrect setup, the game will not simply stop the activity.

Instead:

```
Incorrect Setup
      ↓
System Detects Problem
      ↓
Show Feedback
      ↓
Highlight Problem Area
      ↓
Player Investigates
      ↓
Player Changes Part / Connection
      ↓
Test Again
```

This makes troubleshooting part of the learning process.

For example, the game may display:

> **“Check the connection between these two parts.”**

or:

> **“A required part is missing.”**

or:

> **“This part is placed in the wrong position.”**

The feedback system will help beginners learn through experimentation.

10.8 Visual Device Response

When the required configuration is successfully completed, the game can provide a visual response such as:

- Light turning ON
- Fan rotating
- Button producing an action
- Display activating
- Movement starting
- Indicator changing state
- A complete device becoming operational

This immediate response helps the player understand:

> **“What I built caused this function to happen.”**

10.9 Simplified Simulation Approach

The first version of the game will use a** simplified simulation model**.

The objective is to make the system:

- Easy to understand
- Easy to develop
- Fast to test
- Suitable for beginners
- Easy to expand

The game will initially focus on **functional correctness rather than complete physical accuracy**.

For example, the system may define:

```
Required Parts
        +
Correct Placement
        +
Correct Connections
        +
Required Conditions
        ↓
Device Works
```

This does not mean that real-world principles are ignored. Rather, the project will introduce complexity gradually.

10.10 Future Simulation Expansion

As the project develops, the simulation model can become more detailed.

Future versions may introduce:

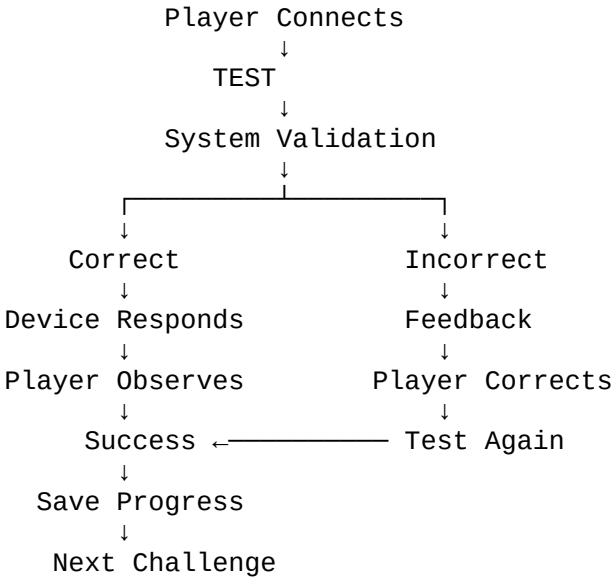
- More realistic device behavior
- More detailed internal sections
- Advanced connection rules
- Fault conditions
- Component failures
- Troubleshooting scenarios
- More realistic visual responses
- More accurate electrical behaviour

Therefore, the initial working model will provide the foundation for a progressively more detailed simulation system.

10.11 Complete Working Model

The overall working principle of the game can be summarized as:

```
REAL-WORLD DEVICE CONCEPT
            ↓
     Digital Device
            ↓
     Parts & Sections
            ↓
      Player Explores
            ↓
     Player Assembles
            ↓
```



The working model is therefore based on a simple principle:

> **The player does not merely read how a device works; the player builds, tests and observes the device working.**

---

11. TESTING

Testing will be performed in **two major stages** during the development of the game.

The purpose of dividing testing into two stages is to first verify whether the developed system is functioning correctly and then perform a second round of testing after identifying and correcting the problems found during the initial testing process.

The overall testing cycle will be:

> **Develop → Test Stage 1 → Identify Problems → Fix Problems → Test Stage 2 → Verify Final Performance**

11.1 TESTING STAGE 1 — INITIAL FUNCTIONAL TESTING

The first stage of testing will be performed after the initial implementation of a playable version.

The primary purpose of Stage 1 is to determine whether the basic game systems are **working correctly and interacting with each other as expected**.

This stage will focus mainly on technical and functional correctness rather than detailed user feedback.

11.1.1 Game Start Testing

The basic game startup process will be tested.

The following will be checked:

- Does the game open correctly?
- Does the main menu appear?
- Can the player start the game?
- Does the tutorial/activity open correctly?
- Does the selected activity load successfully?
- Are there any startup crashes or major errors?

11.1.2 Part Interaction Testing

The interactive parts used in the game will be tested individually.

The following actions will be checked:

```
Select Part
    ↓
Pick Up Part
    ↓
Drag Part
    ↓
Place Part
    ↓
Move / Reposition Part
    ↓
Remove Part
```

The tester will verify that parts:

- Can be selected
- Can be moved
- Can be placed
- Remain visible
- Do not disappear unexpectedly
- Respond correctly to player input

11.1.3 Connection and Assembly Testing

The connection and assembly system will be tested to make sure that parts can interact correctly.

The following will be checked:

- Can required parts be connected?
- Are connection points recognized?
- Can incorrect connections be detected?
- Can the player change an incorrect connection?
- Does the system recognize when the assembly is complete?

11.1.4 Device Response Testing

After assembly, the game will be tested to determine whether the device provides the expected response.

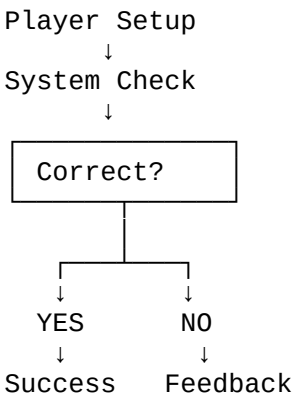
Examples:

Action	Expected Result
Correct lamp setup	Lamp turns ON
Correct fan setup	Fan starts rotating
Correct button setup	Expected action occurs
Correct device assembly	Device becomes operational
Incorrect setup	Device does not produce the expected result

The purpose is to verify that the **game's internal rules and visible device behaviour are working correctly**.

11.1.5 Validation Testing

The validation system will be tested with both correct and incorrect setups.



The system will be checked for:

- Missing parts
- Incorrect parts
- Incorrect placement
- Wrong connections
- Incomplete assembly
- Correct complete assembly

11.1.6 UI Testing

The main interface will be tested to ensure that all basic controls function correctly.

The following will be checked:

- Main menu
- Objective section
- Parts panel
- Test button
- Reset button
- Feedback section
- Progress display
- Activity selection
- Pause/menu controls

The tester will verify that buttons respond correctly and that important interface elements are visible and usable.

11.1.7 Save and Progress Testing

The local progress system will be tested.

The basic flow will be:

```
Complete Activity
       ↓
Save Progress
       ↓
Close Game
       ↓
Open Game Again
       ↓
Load Saved Data
       ↓
Check Previous Progress
```

The following will be checked:

- Activity completion is saved
- Progress remains after restarting the game
- Completed activities remain unlocked
- Settings are preserved where applicable
- Save and load operations work correctly

11.1.8 Basic Performance Testing

The first version will also undergo basic performance testing.

The following will be observed:

- Game loading time
- Frame rate
- Input responsiveness
- Visible lag
- Unexpected crashes
- Major visual problems
- Unusual memory usage

The purpose at this stage is to identify **major technical problems** rather than perform detailed optimization.

11.2 RESULT OF TESTING STAGE 1

All problems identified during Stage 1 will be documented.

The problems may include:

```
Gameplay Bug
UI Bug
Interaction Bug
Connection Bug
Validation Bug
Device Response Bug
Save/Progress Bug
Performance Problem
Visual Problem
```

Each identified problem will be recorded with:

Information	Description
Problem	What went wrong
Location	Where it occurred
Cause	Possible reason
Priority	Low / Medium / High
Required Fix	What needs to be changed
Status	Open / Fixed

12. IMPLEMENTATION AND INTEGRATION

12.1 Implementation Process

The implementation stage will focus on bringing the major game systems together into a functional and integrated game.

The implementation process will include:

```
Game Concept
     ↓
Game Environment
     ↓
Interactive Parts
     ↓
Device Structure
     ↓
Player Interaction
```

```
      ↓
Assembly / Connection System
      ↓
Validation System
      ↓
Device Response
      ↓
UI / Feedback
      ↓
Progress / Save System
      ↓
Integrated Playable Version
```

12.2 Major Systems to be Implemented

The playable version of the game will include the following major systems:

Game Environment

The basic game environment and interactive workspace will be created in Unreal Engine.

Interactive Parts

The required parts and objects will be created or imported and made interactive.

The player will be able to:

- Select parts
- Move parts
- Place parts
- Rotate parts where required
- Connect parts
- Remove incorrect parts

Device System

The device system will define how different parts are organized and how they contribute to the function of the device.

Interaction System

The Drag-and-Drop interaction system will connect the player's actions with the game environment.

Assembly and Connection System

The system will allow the player to assemble parts and create the required relationships between them.

Validation System

The validation system will determine whether the player's setup satisfies the requirements of the current activity.

Device Response

When the setup is correct, the corresponding device or function will provide an appropriate visual or interactive response.

User Interface

The required UI elements will be integrated into the playable experience, including:

- Main menu
- Activity/level information
- Objective panel
- Parts panel
- Test button
- Reset button
- Feedback area
- Progress indicator

Progress and Save System

The SaveGame system will be integrated to store the player's progress and completed activities.

Audio and Visual Feedback

Basic sound effects, animations and visual responses will be integrated to make the interaction understandable and engaging.

12.3 Correction and Improvement After Testing Stage 1

The problems identified during Testing Stage 1 will be addressed during this implementation stage.

The correction process will follow:

```
Testing Stage 1
      ↓
Problems Identified
      ↓
Problems Classified
      ↓
Cause Investigated
      ↓
Required Changes Implemented
      ↓
Affected Systems Updated
      ↓
Integrated Version Created
      ↓
Testing Stage 2
```

Problems may include:

- Interaction errors
- Incorrect connections
- Validation errors

- Device response problems
- UI problems
- Save/progress problems
- Visual issues
- Performance problems

12.4 System Integration

After individual corrections are implemented, all major systems will be integrated and tested as a combined game experience.

The integration structure will be:

```
Interactive Parts
       ↓
Player Interaction
       ↓
Assembly / Connection
       ↓
Validation
       ↓
Device Response
       ↓
Feedback
       ↓
Activity Completion
       ↓
Progress Saving
```

The purpose of integration is to ensure that individual systems do not work independently only, but also operate correctly when combined.

12.5 Playable Version

At the completion of this stage, the project should have a stable **integrated playable version** suitable for Testing Stage 2.

The version should contain the core features required for gameplay:

- Main menu
- Tutorial / basic guidance
- Activity selection
- Interactive workspace
- Parts and device objects
- Drag-and-drop interaction
- Assembly and connection
- Validation
- Device response
- Feedback system
- Progress tracking

- Save system
- Basic audio and visual feedback

The focus at this stage is to ensure that the complete gameplay cycle can be experienced from beginning to end.

12.6 Modular Implementation

The implementation will follow a modular approach so that individual systems can be modified without requiring the entire game to be rebuilt.

The major modules will include:

```
Game
│
├── User Interface
├── Parts
├── Devices
├── Interaction
├── Assembly
├── Validation
├── Feedback
├── Progress
└── Save System
```

This structure will make future updates and expansion easier.

New devices, parts and activities can therefore be introduced by extending existing systems rather than redesigning the complete project.

12.7 Output of the Implementation Stage

The expected output of this stage will be:

> **A corrected, integrated and playable version of the game that is ready for Testing Stage 2.**

The implementation stage therefore acts as the bridge between the first technical test and the second verification cycle.

The development cycle can be represented as:

```
      INITIAL DEVELOPMENT
               ↓
      TESTING STAGE 1
               ↓
      FIND PROBLEMS
               ↓
  IMPLEMENTATION & CORRECTION
               ↓
      SYSTEM INTEGRATION
               ↓
        PLAYABLE BUILD
               ↓
      TESTING STAGE 2
```

```
          ↓
USER INTERACTION TEST
          ↓
       FEEDBACK
          ↓
FURTHER IMPROVEMENT
```

This approach ensures that the second testing stage is performed on an **updated and integrated version** rather than on the same version that was tested initially.

---

13. TESTING STAGE 2 — RE-TESTING AND CORRECTION TESTING

Testing Stage 2 will begin **after the problems identified during Stage 1 have been corrected**.

The purpose of Stage 2 is to verify two things:

1. The previously identified problems have actually been fixed.
2. The fixes have not created new problems in other parts of the game.

Therefore, Stage 2 will include both **re-testing and regression testing**.

13.1 Bug Re-Testing

Every problem identified during Stage 1 will be tested again.

For example:

```
Stage 1
   ↓
Connection Problem Found
   ↓
Developer Fixes Problem
   ↓
Stage 2
   ↓
Test Same Connection Again
   ↓
Problem Fixed?
   /       \
 YES        NO
 ↓           ↓
Close      Fix Again
Issue
```

13.2 Regression Testing

After a problem is fixed, related features will also be tested to ensure that the correction did not break another part of the game.

For example:

> If the connection system is changed, the tester will not only test the original faulty connection but also verify other valid connections.

Similarly:

> If the save system is modified, previously completed activities and progress loading will also be tested.

This ensures that individual fixes do not introduce new errors.

13.3 Complete Gameplay Re-Test

After individual problems are corrected, complete activities will be played again from beginning to end.

```
Start Game
    ↓
Select Activity
    ↓
Read Objective
    ↓
Select Parts
    ↓
Assemble
    ↓
Connect
    ↓
Test
    ↓
Observe Result
    ↓
Complete Activity
    ↓
Save Progress
    ↓
Load Again
    ↓
Continue to Next Activity
```

The purpose is to verify that all systems work together correctly.

13.4 UI and Interaction Re-Test

The interface and interaction system will be tested again after corrections.

The tester will verify:

- Buttons still work
- Parts can still be moved
- Connections still work
- Feedback still appears
- Objectives display correctly
- Test and reset functions still operate
- No UI elements overlap or disappear

13.5 Device Behaviour Re-Test

Every corrected device or activity will be tested again to verify that the expected response occurs.

The testing will confirm:

```
Correct Setup
      ↓
Validation
      ↓
Expected Device Response
      ↓
Success
```

and:

```
Incorrect Setup
      ↓
Validation
      ↓
Feedback
      ↓
Correction
```

13.6 Final Stability Check

After all identified problems have been addressed, a final stability check will be performed.

The game will be observed for:

- Unexpected crashes
- Major bugs
- Broken interactions

- Incorrect validation
- Save/load problems
- Device response problems
- UI problems
- Performance issues

If no major problems remain, the tested version can proceed toward the next stage of development, including **User Interaction Testing and Feedback Collection**.

13.7 TESTING CYCLE

The complete testing process can be summarized as:

```
Development
    ↓
Testing Stage 1
    ↓
Find Problems
    ↓
Document Problems
    ↓
Fix Problems
    ↓
Testing Stage 2
    ↓
Re-Test Fixed Features
    ↓
Regression Testing
    ↓
Complete Gameplay Test
    ↓
Final Stability Check
    ↓
Ready for User Testing
```

The testing methodology therefore follows a simple principle:

Test → Find → Fix → Test Again → Verify

This two-stage approach will help ensure that the game is technically stable before it is exposed to actual users for **User Interaction Testing and Feedback**.

---

14. UPDATION

The game will follow an **iterative development and improvement model**.

After the completion of Testing Stage 2, User Interaction Testing and Feedback Collection, the results will be analyzed to identify areas where the game can be improved, expanded or refined.

The updation process will ensure that the project continues to develop based on **technical performance, user experience, learning effectiveness and future content requirements**.

The overall improvement cycle will be:

```
Testing Stage 2
      ↓
User Interaction Testing
      ↓
Feedback Collection
      ↓
Problem / Improvement Identification
      ↓
Prioritize Changes
      ↓
Implement Updates
      ↓
Test Updated Features
      ↓
Release Improved Version
      ↓
Repeat Cycle
```

14.1 Areas of Updation

The project can be updated in several major areas.

1. New Devices and Activities

New electrical and electronic devices can be introduced to expand the learning experience.

Future content may include:

- New household devices
- New control systems
- New electronic assemblies
- More complex internal structures
- Additional interactive device sections

The modular architecture will allow new content to be added without rebuilding the complete game.

---

2. New Parts and Objects

Additional parts can be introduced as the learning scope expands.

Each new part can include:

- 3D model
- Name
- Function
- Connection points
- Visual behaviour
- Associated sound

This will allow the game to continuously expand its library of interactive objects.

---

3. Gameplay and Interaction Improvements

Based on testing and user feedback, the interaction system can be improved.

Possible improvements include:

- Smoother Drag-and-Drop
- Better object placement
- Improved connection indicators
- Better camera controls
- Easier part selection
- Improved object highlighting
- Improved reset and correction options

The objective is to make the interaction more natural and intuitive.

---

4. Learning Experience Improvements

The game will be updated according to how easily users understand the activities.

Possible improvements include:

- Simpler instructions
- Visual hints
- Step-by-step guidance

- Improved explanations
- Better error messages
- Interactive demonstrations

---

5. Visual and Audio Improvements

The game's presentation can be improved through:

- Better 3D/2D models
- Improved animations
- More realistic device responses
- Better visual effects
- Improved lighting
- New sound effects
- Improved interaction sounds

These updates will improve engagement without changing the core learning mechanism.

---

7. UI / User Experience Improvements

The interface can be refined based on actual user behaviour.

Possible updates include:

- Improved menu layout
- Better objective display
- Better part organization
- Improved buttons
- Improved feedback messages
- Better progress indicators
- Improved navigation
- Reduced unnecessary information

The UI will continue to follow the principle:

Simple to Understand → Easy to Interact → Clear to Learn

---

8. Validation and Device Behaviour Improvements

The internal validation system can be improved as more complex activities are introduced.

Future improvements may include:

- More detailed checking
- Better error identification
- More specific feedback
- Multiple valid configurations where appropriate
- Fault simulation
- Troubleshooting challenges
- More detailed device responses

This will allow the game to move gradually from simple configuration checking toward more advanced system interaction.

---

9. Performance Optimization

As the game becomes larger, performance will be monitored and optimized.

Possible improvements include:

- Faster loading
- Reduced memory usage
- Better frame rate
- Optimized 3D/2D assets
- Reduced unnecessary processing
- Improved animation performance
- Better handling of large scenes

Performance optimization will become increasingly important as the number and complexity of interactive objects increase.

14.2 Content Expansion

The game will be designed so that additional content can be introduced continuously.

The content structure can expand as:

```
Existing Activities
        +
New Parts
        +
New Devices
        +
New Challenges
        +
New Interactions
```

```
        ↓
Expanded Learning Experience
```

This allows the project to evolve beyond the initial playable version.

14.3 Feedback-Based Updates

User feedback will be treated as an important source of future improvements.

For example:

```
User Feedback
      ↓
Identify Common Problem
      ↓
Determine Cause
      ↓
Design Improvement
      ↓
Implement Change
      ↓
Test Change
      ↓
Add to Future Version
```

Changes will be prioritized according to their impact on:

- Usability
- Learning
- Gameplay
- Stability
- Performance

14.4 Future Version Development

The project can be developed through multiple versions.

```
Version 1
Basic Interactive Prototype
        ↓
Version 2
Improved Gameplay + More Content
        ↓
Version 3
Advanced Device Interaction
        ↓
Version 4
Expanded System Understanding
        ↓
Future Versions
More Devices + More Advanced Systems
```

Each version will be based on the results and requirements identified during the previous development cycle.

14.5 Final Objective of Updation

The purpose of continuous updation is not simply to increase the amount of content.

The primary objective is to make the game:

> **More understandable, more interactive, more stable, more engaging and more effective as a learning experience.**

The project will therefore follow a continuous cycle of:

> **Build → Test → Correct → Observe → Improve → Expand**

This iterative approach will allow the game to evolve from an initial prototype into a larger and more refined interactive learning platform.

15. USER INTERACTION TEST

User Interaction Testing will be conducted **after the completion of Testing Stage 1, Implementation and Correction, Testing Stage 2, and the initial Updation process**.

Unlike the previous technical testing stages, this phase will involve **actual target users or customers** who represent the intended audience of the game.

The purpose of this testing is to evaluate the game from a real user's perspective and determine whether the completed version provides an understandable, engaging and effective learning experience.

The overall development and testing sequence will therefore be:

```
Initial Development
        ↓
Testing Stage 1
        ↓
Problem Identification
        ↓
Implementation & Correction
        ↓
Testing Stage 2
        ↓
Updation
        ↓
Stable / Improved Version
        ↓
ACTUAL USER TESTING
        ↓
User Observation
        ↓
Feedback Collection
        ↓
Final Improvements
```

15.1 Purpose of Actual User Testing

The main objective of this phase is to determine whether the game works effectively **for the people for whom it is being developed**, rather than only confirming that the software works technically.

The testing will evaluate:

- Whether users understand the purpose of the game.
- Whether users can understand the instructions without continuous assistance.
- Whether users can interact with the game naturally.
- Whether users understand how to assemble and connect parts.
- Whether users understand the result produced by the device.
- Whether users find the experience engaging.
- Whether the learning progression feels understandable.
- Whether users are interested in continuing to the next activity.
- Whether the game successfully communicates how different parts contribute to a device's function.

15.2 Target Users / Customers

The testing will be conducted with **actual target users or representative customers** of the product.

Depending on the final target market, these may include:

- Children or young learners
- Beginners interested in electronics
- Students
- Parents evaluating educational value
- Teachers or educators
- Other intended users of the product

The exact user group will be selected according to the final target audience of the game.

The objective is to test the game with users who are representative of the people expected to use or purchase the product.

15.3 User Testing Process

The actual user test will follow a controlled but natural gameplay process.

```
Select Target User
```

```
        ↓
Briefly Explain Basic Controls
        ↓
User Starts the Game
        ↓
User Reads the Objective
        ↓
User Explores the Activity
        ↓
User Selects Parts
        ↓
User Places / Assembles Parts
        ↓
User Connects Parts
        ↓
User Tests the Setup
        ↓
User Observes Device Response
        ↓
User Attempts to Complete Activity
        ↓
Observe User Behaviour
        ↓
Collect Feedback
        ↓
Analyze Results
```

The tester will provide only the minimum necessary instructions so that the actual usability of the game can be observed.

15.4 What Will Be Observed?

During the test, the development team will observe how the actual user interacts with the game.

Understanding

- Does the user understand the objective?
- Does the user know what needs to be done?
- Can the user understand the meaning of the available parts?
- Does the user know what to do next without being told?

Interaction

- Can the user select a part easily?
- Is Drag-and-Drop intuitive?
- Can the user place a part correctly?
- Can the user connect parts without confusion?
- Can the user correct a mistake?

Device Exploration

- Does the user show interest in exploring the inside of a device?
- Can the user identify important parts or sections?
- Does the user understand the relationship between different parts?
- many more....

Result Understanding

- Does the user notice when the device starts working?
- Does the user understand why the device responded?
- Can the user connect their actions with the final result?
- many more....

Engagement

- Does the user remain interested?
- Does the user voluntarily explore?
- Does the user want to try again after making a mistake?
- Does the user want to continue to another activity?
- many more....

15.5 User Tasks

The actual customer/user may be asked to perform selected activities without detailed step-by-step assistance.

For example:

```
Open the Game
      ↓
Select an Activity
      ↓
Read the Objective
      ↓
Identify Required Parts
      ↓
Build / Assemble the Setup
      ↓
Test the Setup
      ↓
Observe the Result
      ↓
Complete the Activity
      ↓
Move to the Next Activity
```

The exact activity will depend on the development stage and the type of device being tested.

15.6 Measuring User Experience

The user experience will be evaluated through both **observation and direct feedback**.

Observation

The team will record:

- Time required to understand the objective
- Number of mistakes
- Number of times assistance was required
- Difficulty in finding parts
- Difficulty in assembling the device
- Difficulty in understanding feedback
- Whether the user completes the activity independently

User Feedback

After the session, users may be asked questions such as:

- Was the game easy to understand?
- Was anything confusing?
- Were the parts easy to identify?
- Was the interaction comfortable?
- Did you understand what you were building?
- Did you understand why the device worked?
- Which part of the game did you enjoy most?
- Which part was difficult?
- Did the game make you curious about how the real device works?
- Would you like to play another activity?
- Would you recommend this type of learning game to someone else?
- many more....

15.7 User Testing Evaluation

The collected results can be documented using a structured evaluation table.

Evaluation Area	What Will Be Observed
Objective Understanding	Can the user understand what needs to be done?
Part Recognition	Can the user identify required parts?
Interaction	Can the user use Drag-and-Drop comfortably?
Assembly	Can the user place and connect parts?
Device Understanding	Does the user understand how parts contribute to the device?
Feedback	Can the user understand success and error messages?
Learning	Does the activity communicate the intended concept?
Difficulty	Is the activity too easy, suitable or too difficult?
Engagement	Does the user remain interested?
Independence	Can the user complete the activity without continuous assistance?

15.8 Customer Acceptance Indicators

Important indicators will include:

> **Understanding + Engagement + Learning Value + Usability + Interest in Continued Use**

15.9 Feedback-to-Improvement Process

The feedback collected from actual users will become an important input for the next development cycle.

```
Actual User Testing
        ↓
Observe User Behaviour
        ↓
Collect Feedback
        ↓
Identify Problems / Suggestions
        ↓
Classify Feedback
        ↓
Prioritize Changes
        ↓
Implement Improvements
        ↓
Test Improvements
        ↓
Release Updated Version
```

Feedback may lead to improvements in:

- User interface
- Instructions
- Difficulty
- Device representation
- Interaction
- Feedback system
- Learning content
- Visual design
- Audio
- Gameplay flow

15.10 Final Objective of User Interaction Testing

The ultimate purpose of this phase is to answer a question that technical testing alone cannot answer:

> **“Does the game actually work for the real user?”**

Actual user testing will be used to verify that the game is:

> **Easy to Understand → Easy to Use → Interesting to Explore → Educational to Experience**

The findings from this phase will be used for the final refinement of the product and for planning future versions, content and features.

---

16. FEEDBACK

Feedback will be collected **after the User Interaction Testing phase**, when the game has already passed the initial technical testing, correction, second testing stage and initial updation process.

The purpose of collecting feedback is to understand how the **actual target users or customers** experience the game and to identify improvements that may not have been discovered during internal testing.

The feedback process will focus on five major areas:

> **User Experience → Usability → Learning → Engagement → Product Improvement**

16.1 Feedback Collection Process

The feedback process will follow:

```
Actual User Testing
        ↓
User Experience Observed
        ↓
Questions / Feedback Collected
        ↓
Problems and Suggestions Identified
        ↓
Feedback Categorized
        ↓
Important Issues Prioritized
        ↓
Required Improvements Planned
        ↓
Future Updation
```

Feedback will be collected through:

- Direct user discussion
- Short questionnaires
- Observation during gameplay
- User ratings
- Suggestions and comments

The method of collection may vary according to the target audience.

16.2 Positive Feedback

Positive feedback will identify the aspects of the game that users find successful.

Users may consider the game:

- Interesting
- Interactive
- Easy to understand
- Enjoyable
- Visually attractive
- Educational
- Satisfying to play
- Easy to explore
- Motivating to continue

Positive feedback will help identify the features that should be maintained or expanded in future versions.

16.3 Usability Feedback

Usability feedback will focus on how easily users can operate and understand the game.

The following areas will be evaluated:

- Main menu
- Instructions
- Buttons
- Drag-and-Drop interaction
- Part selection
- Part placement
- Assembly process
- Connection process
- Test/Check system
- Feedback messages
- Navigation
- Progress system

Examples of feedback may include:

> “The parts were easy to select.”
>
> “The instructions were confusing.”
>
> “The connection points were difficult to identify.”
>
> “The test button was easy to find.”

Such feedback will help improve the overall user experience.

16.4 Learning Feedback

The evaluation will focus on whether users understand:

- What different parts are used for
- How different parts contribute to a device
- How parts are arranged inside a device

- How different sections work together
- How an input can produce a result
- Why a device works after the correct setup
- How problems can be identified and corrected
- How simple systems can be combined into more complex systems

The purpose is not simply to check whether users completed an activity, but whether they **understood something from completing it**.

16.5 Engagement Feedback

Since the project is designed as a game-based learning experience, user engagement is an important part of the feedback process.

The following will be evaluated:

- Did the user enjoy the activity?
- Did the user remain interested?
- Did the user want to explore more?
- Did the user want to try again after making a mistake?
- Did the user want to continue to another activity?
- Which activities were most interesting?
- Which activities felt repetitive or less engaging?
- many more....

This feedback will help maintain a balance between **learning and entertainment**.

16.6 Difficulty Feedback

Users will also be asked about the difficulty of the activities.

The feedback can help determine whether an activity is:

> **Too Easy → Appropriate → Too Difficult**

For example:

```
User Attempts Activity
        ↓
Observe Difficulty
        ↓
Collect User Opinion
        ↓
Compare With Expected Difficulty
        ↓
Adjust if Required
```

If many users find an activity too difficult, the instructions, hints or task complexity may be improved.

If an activity is consistently too easy, additional challenge can be introduced.

16.7 Device and Visual Feedback

Because the game focuses on understanding how real-world devices work internally, users will also provide feedback on the representation of those devices.

The following will be evaluated:

- Are the device parts recognizable?
- Does the device look understandable?
- Are internal sections easy to explore?
- Are animations helpful?
- Does the device response clearly communicate what happened?
- Does the game make the user curious about the real-world device?
- many more....

This feedback will help improve the connection between the **virtual experience and the real-world learning concept**.

16.8 Customer Feedback

Since the testing phase involves actual target users or customers, their broader opinion of the product will also be considered.

Possible questions include:

- Would you use this game again?
- Would you recommend it to others?
- Do you think it is useful for learning?
- Does the concept feel different from traditional learning methods?
- Which features would you like to see added?
- Which features should be improved?
- Would you be interested in more devices and activities?

This feedback can also help evaluate the product's future direction.

16.9 Feedback Evaluation

Collected feedback can be organized into a structured format.

Feedback Category	What Will Be Evaluated
Positive	What users liked
Usability	Ease of interaction and navigation
Learning	What users understood
Engagement	Interest and enjoyment
Difficulty	Appropriate challenge level
Visual	Device and interface presentation
Device Understanding	Understanding of internal working
Suggestions	Features users want in future
Problems	Issues requiring correction

16.10 Feedback-Based Improvement

The collected feedback will not only be recorded but will also be converted into actionable improvements.

The process will be:

```
Feedback
   ↓
Analyze
   ↓
Identify Common Issues
   ↓
Separate Major and Minor Issues
   ↓
Prioritize
   ↓
Plan Improvements
   ↓
Implement Changes
   ↓
Test Changes
   ↓
Include in Future Version
```

For example:

```
User Feedback:
"Parts are difficult to identify."

        ↓

Improvement:
Increase visual difference +
Add labels / highlighting

        ↓

New Version

        ↓

Retest with Users
```

16.11 Feedback Documentation

All important feedback will be documented for future reference.

A feedback record may contain:

Field	Description
User Type	Student / Child / Parent / Teacher / Other
Feature	Part of the game being discussed
Feedback	User's comment
Category	Usability / Learning / Engagement / etc.
Priority	Low / Medium / High
Proposed Action	Required improvement
Status	Pending / In Progress / Completed

This will provide a structured record of how user feedback influences the development of the project.

16.12 Final Purpose of Feedback

The primary purpose of the feedback process is to ensure that the project continues to improve based on **real user experience rather than assumptions made only by the development team**.

The feedback stage will help answer:

> **“What do users actually like, understand, struggle with and want from the game?”**

The final improvement cycle will therefore follow:

> **Test → Observe → Ask → Analyze → Improve → Re-Test**

Through this process, the project can gradually become more **usable, engaging, educational and valuable to its intended users**.

---

17. CONCLUSION

The proposed project, **“From Components to Systems,”** aims to transform the way electrical and electronic devices are explored and understood by introducing an **interactive, practical and game-based learning experience**.

The central idea of the project is to enable users to understand how real-world devices are made from smaller parts and functional sections, how these parts are connected, and how they work together to produce the final function of a device.

Instead of depending primarily on theoretical explanations, the game allows users to **explore, assemble, connect, test and observe** virtual representations of electronic devices. This interaction-based approach is intended to make technical concepts easier to understand, particularly for beginners and young learners.

The project follows a progressive learning structure in which the complexity of the tasks increases gradually. Users begin with simple interactions and functional relationships and progressively move toward more complex devices and larger interconnected systems. The objective is to develop an intuitive understanding of the relationship between:

Parts → Connections → Functions → Devices → Larger Systems

The proposed technology stack, centered around **Unreal Engine, Blueprints, C++, Blender and supporting development tools**, provides the foundation for creating an interactive and expandable game environment. The initial architecture will remain local and lightweight, avoiding unnecessary backend complexity while maintaining the flexibility to introduce online features in future versions.

The development process will follow a structured testing methodology. **Testing Stage 1** will identify initial technical and functional problems, followed by **Implementation and Correction**, and then **Testing Stage 2** to verify that the identified issues have been resolved without introducing new problems.

After the internal testing and updation process, the project will be evaluated with **actual target users or customers**. User Interaction Testing and Feedback Collection will provide real-world insights into usability, engagement, learning value, difficulty and overall product acceptance. These findings will be used to further refine and improve future versions of the game.

The project is therefore not intended to be only a game or only an electronics simulator. Its broader objective is to create an **interactive digital learning environment** where users can develop curiosity, practical understanding and problem-solving ability by discovering how things work through direct experimentation.

The long-term vision is to continuously expand the range of devices, systems, interactions and learning challenges so that the project can evolve from a simple prototype into a more comprehensive interactive platform for understanding technology and electronic systems.

The overall philosophy of the project can be summarized as:

Explore → Build → Connect → Test → Observe → Understand → Progress

Through this approach, **“From Components to Systems”** seeks to make the understanding of electronic devices more **interactive, accessible, engaging and practical**, while providing a foundation for continued technological and educational development.
